# Experiment No: 06

**Student Name:** Asmit Prabhakar
**UID:** 23BCS10063
**Branch:** BE-CSE
**Section/Group:** KRG_2B
**Semester:** 6th
**Date of Performance:** 04/03/2026
**Subject Code:** 23CSH-314
**Subject Name:** System Design

## Aim:

To design a highly scalable video streaming platform similar to YouTube that supports heavy video uploading, adaptive bitrate streaming, and real-time social interactions with high availability and low latency.

## Objective:

- Understand the architecture of a massive-scale media streaming system.
- Identify functional requirements such as direct-to-cloud uploads and personalized feeds.
- Identify non-functional requirements including CDN caching, high throughput, and latency constraints.
- Analyze the decoupling of synchronous API requests using event-driven stream processing.
- Design REST APIs for video management, search, and user interactions.

## Procedure:

- Studied real-world video streaming architectures and content delivery networks.
- Identified core entities such as Users, Videos, Interactions, and View Histories.
- Analyzed the Pre-Signed URL pattern to bypass API Gateway bottlenecks for large file uploads.
- Collected functional and non-functional requirements for media delivery.
- Designed APIs for video processing, metadata retrieval, and social engagement.
- Evaluated streaming protocols (HLS/DASH) and separated thumbnail storage for optimization.
- Analyzed stream processing (Apache Flink/Kafka) for real-time, high-volume view aggregation.

## Functional Requirements:

- System should allow users to register, log in, and manage accounts.
- System should support uploading large raw video files efficiently.
- System should stream video smoothly using Adaptive Bitrate Streaming based on network conditions.
- System should support real-time social interactions (likes, dislikes, comments).
- System should allow users to search for videos by title or metadata.
- System should generate a personalized home page feed based on watch history.

## Core Entities of the System:

- Users
- Videos

- Likes and comments
- Video feed

## API Design:

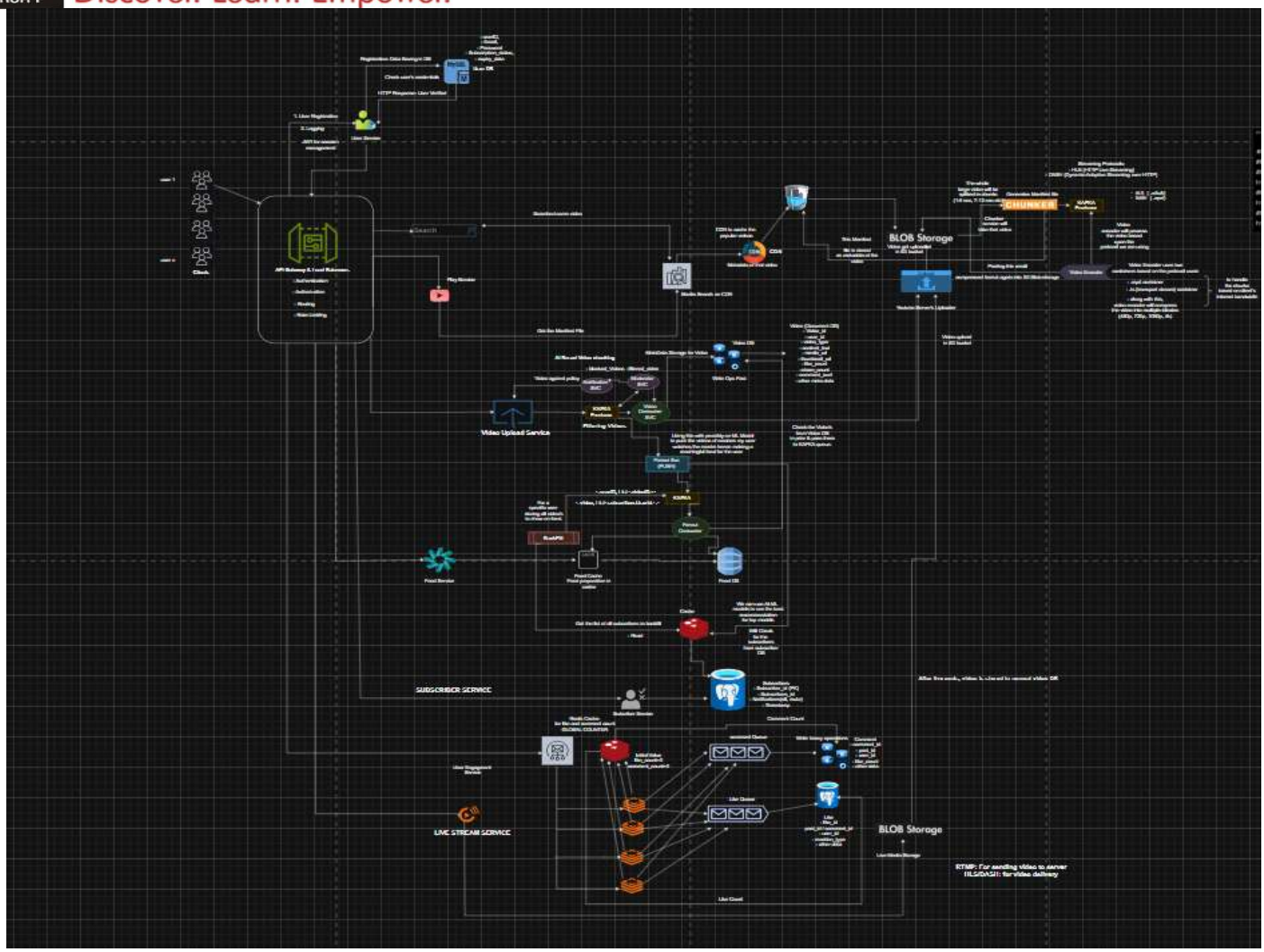
- Authentication API
  - Register: POST /api/v1/auth/register
  - Login: POST /api/v1/auth/login
- Video & Feed API
  - Initialize Upload: POST /api/v1/videos/upload/init (Returns S3 Pre-Signed URL)
  - Get Video Metadata: GET /api/v1/videos/{video_id}
  - Get Personalized Feed: GET /api/vl/feed (Pagination Supported)
  - Search Videos: GET /api/v1/search?q={query}
- Interaction API
  - Like Video: POST /api/v1/videos/{video_id}/like
  - Add Comment: POST /api/v1/videos/{video_id}/comments
  - Get Comments: GET /api/v1/videos/{video_id}/comments

## Non-Functional Requirements:

- System must handle extremely high throughput for concurrent uploads and millions of simultaneous views.
- High Availability must be prioritized, ensuring no single point of failure.
- System should follow an Eventual Consistency model for social metrics (view counts, likes) to optimize database write loads.
- Video delivery latency should be minimized via Edge Caching and a Global CDN.
- Transcoding pipeline must be horizontally scalable to handle viral upload spikes.

## High Level Design: -

The system consists of Client Applications, an API Gateway, and specialized microservices (Auth, Upload, Video Metadata, Feed, and Interactions). Heavy uploads bypass the backend via Pre-Signed URLs directly to Object Storage. Asynchronous transcoding pipelines process videos into HLS/DASH segments, which are delivered globally via a CDN. High-volume interactions are managed using Redis (Likes), NoSQL clusters (Comments), and a Kafka-to-Apache Flink stream processing pipeline for accurate view aggregation.




## Outcome:

- Designed a highly scalable, event-driven video streaming architecture.
- Identified critical functional and non-functional requirements for media delivery.
- Designed REST APIs and decoupled asynchronous processing pipelines.
- Understood the application of CDNs, adaptive bitrate streaming, and stream processing for massive-scale systems.